Latest updates: https://dl.acm.org/doi/10.1145/3690624.3709263

RESEARCH-ARTICLE

MGS3: A Multi-Granularity Self-Supervised Code Search Framework

RUI LI, University of Science and Technology of China, Hefei, Anhui, China

JUNFENG KANG, University of Science and Technology of China, Hefei, Anhui, China

QI LIU, University of Science and Technology of China, Hefei, Anhui, China

LIYANG HE, University of Science and Technology of China, Hefei, Anhui, China

ZHENG ZHANG, University of Science and Technology of China, Hefei, Anhui, China

YUNHAO SHA, University of Science and Technology of China, Hefei, Anhui, China

View all

Open Access Support provided by:

University of Science and Technology of China

PDF Download
3690624.3709263.pdf
31 March 2026
Total Citations: 1
Total Downloads: 146

Published: 20 July 2025

Citation in BibTeX format

KDD '25: The 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining
August 3 - 7, 2025
Toronto ON, Canada

Conference Sponsors:
SIGMOD
SIGKDD

KDD '25: Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1 (July 2025)
https://doi.org/10.1145/3690624.3709263
ISBN: 9798400712456

MGS3: A Multi-Granularity Self-Supervised Code Search Framework

Rui Li
State Key Laboratory of Cognitive Intelligence, University of Science and Technology of China
Hefei, China
ruili2000@mail.ustc.edu.cn

Junfeng Kang
State Key Laboratory of Cognitive Intelligence, University of Science and Technology of China
Hefei, China
kangjf@mail.ustc.edu.cn

Qi Liu*
State Key Laboratory of Cognitive Intelligence, University of Science and Technology of China & Institute of Artificial Intelligence, Hefei Comprehensive National Science Center
Hefei, China
qiliuql@ustc.edu.cn

Liyang He
State Key Laboratory of Cognitive Intelligence, University of Science and Technology of China
Hefei, China
heliyang@mail.ustc.edu.cn

Zheng Zhang
State Key Laboratory of Cognitive Intelligence, University of Science and Technology of China
Hefei, China
zhangzheng@mail.ustc.edu.cn

Yunhao Sha
State Key Laboratory of Cognitive Intelligence, University of Science and Technology of China
Hefei, China
percy@mail.ustc.edu.cn

Linbo Zhu
University of Science and Technology of China & Institute of Artificial Intelligence, Hefei Comprehensive National Science Center
Hefei, China
lbzhu@iai.ustc.edu.cn

Zhenya Huang
State Key Laboratory of Cognitive Intelligence, University of Science and Technology of China & Institute of Artificial Intelligence, Hefei Comprehensive National Science Center
Hefei, China
huangzhy@ustc.edu.cn

Abstract

In the pursuit of enhancing software reusability and developer productivity, code search has emerged as a key area, aimed at retrieving code snippets relevant to functionalities based on natural language queries. Despite significant progress in self-supervised code pre-training utilizing the vast amount of code data in repositories, existing methods have primarily focused on leveraging contrastive learning to align natural language with function-level code snippets. These studies have overlooked the abundance of fine-grained (such as block-level and statement-level) code snippets prevalent within the function-level code snippets, which results in suboptimal performance across all levels of granularity. To address this problem, we first construct a multi-granularity code search dataset called **MGCodeSearchNet**, which contains 536K+ pairs of natural language and code snippets. Subsequently, we introduce a novel **M**ulti-**G**ranularity **S**elf-**S**upervised contrastive learning code **S**earch framework (**MGS**^{3}). First, MGS^3 features a Hierarchical Multi-Granularity Representation module (HMGR), which leverages syntactic structural relationships for hierarchical representation and aggregates fine-grained information into coarser-grained representations. Then, during the contrastive learning phase, we endeavor to construct positive samples of the same granularity for fine-grained code, and introduce in-function negative samples for fine-grained code. Finally, we conduct extensive experiments on code search benchmarks across various granularities, demonstrating that the framework exhibits outstanding performance in code search tasks of multiple granularities. These experiments also showcase its model-agnostic nature and compatibility with existing pre-trained code representation models.

*Qi Liu is the corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
KDD '25, Toronto, ON, Canada
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1245-6/25/08
https://doi.org/10.1145/3690624.3709263

CCS Concepts

• **Information systems → Information retrieval**.

Keywords

Code Search, Information Retrieval, Self-Supervised Learning

ACM Reference Format:
Rui Li, Junfeng Kang, Qi Liu, Liyang He, Zheng Zhang, Yunhao Sha, Linbo Zhu, and Zhenya Huang. 2025. MGS3: A Multi-Granularity Self-Supervised Code Search Framework. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1 (KDD '25), August 3–7, 2025, Toronto, ON, Canada.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3690624.3709263

1 Introduction

Code search aims to retrieve functionally relevant code given a natural language query, facilitating software reuse and enhancing developer productivity. With the rapid expansion of online code repositories such as GitHub[1], current research [8, 27, 28, 41] predominantly focuses on leveraging the rich code data within these repositories for self-supervised code pretraining. Along this line, research work like CodeBERT [7] have attempted to use the masked language modeling (MLM) pretraining task on large-scale code data to improve code representations. However, these methods focus on token-level pretraining tasks, which results in an inability to perform high-quality sequence encoding. To design training objectives more suitable for code search, several studies [4, 27, 29] have turned to contrastive learning for code pre-training in order to enhance the performance of code search.

In practical scenarios, there is often a need for code snippet searches of varying granularity (i.e., statement-level, block-level, and function-level) to satisfy the diverse programming requirements of users. For instance, as shown in Figure 1, a user may wish to save a data file, in which case a function-level *"write_data_to_file"* would meet their needs. Alternatively, a user might simply need to convert the current date and time to the ISO format, and a brief statement of code such as *"date=date.isoformat()"* would suffice for this purpose. However, prior research has predominantly focused on extracting **function-level** code snippets from repositories to construct alignment signals, overlooking the rich repository of fine-grained code snippets. As shown in Figure 1, within a function, there are statement-level code snippets *"date=date.isoformat()"* and block-level code snippets: *"if...*". This motivates us to explore more data-efficient methods for utilizing the diverse granularities of code snippets within repositories, aiming to achieve a **data-efficient code search across all levels of granularity**.

Through in-depth analysis, we have discovered that repositories contain a wide range of natural language comment signals within functions that can be aligned with code snippets at various granularities. For instance, as illustrated in Figure 1, there is an alignment between the inline comment *"check if the file path exists"* and the block-level code snippet *"if…"*, and between the trailing comment *"Get ISO format date"* and the statement-level code snippet *"date=date.isoformat()"*. This observation presents an opportunity to bridge this gap. Specifically, we extract natural language comments from functions in the code repository and pair them with code snippets at different granularities using coding standards and heuristic methods, thereby constructing a code search dataset called **MGCodeSearchNet**. MGCodeSearchNet comprises samples of diverse granularities, including 90.7K function-level pairs, 197K block-level pairs, and 258K statement-level pairs.

[1]https://www.github.com

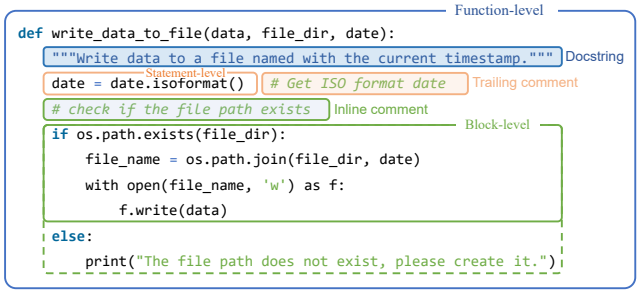


Figure 1: Examples of functions in the repository, which encompass a variety granularities of code snippets, such as those at the function level, block level (e.g., *if...*), and statement level (e.g., *date=date.isoformat()*). Furthermore, the function has three types of natural language comments, including docstrings, inline comments, and trailing comments.

Moreover, enhancing multi-granularity code search with these natural language comments brings the following technical challenges. **On the one hand**, there is syntactic structural information between multi-granularity code snippets. As shown in Figure 1, a statement *"f.write(data)"* exists within a *"with..."* block. Therefore, leveraging these structural relationships to enhance representations across multiple granularities is crucial. **On the other hand**, due to the absence of strict grammatical rules that define the scope of code referred to by comments, this leads to ambiguity in alignment during the pairing process. As shown in Figure 1, the inline comment *"check if the file path exists"* could align with the subsequent *"if..."* block, or it might align with the *"if...else..."* block. This situation exceeds the scope of simple syntactic analysis and requires the integration of both the comment and the semantics of the code snippet for determination.

To tackle the remaining challenges, we introduce a novel **M**ulti-**G**ranularity **S**elf-**S**upervised contrastive learning code **S**earch framework (**MGS**^{3}) for code search models. To leverage structural information to enhance code snippet representations, we introduce a **H**ierarchical **M**ulti-**G**ranularity **R**epresentation (**HMGR**) module that uses the syntactic structural relationships between different fine-grained code snippets for hierarchical representation, aggregating fine-grained representation information into coarse-grained representations for enhanced representation. To address the ambiguity in the alignment process, particularly when natural language comments need to be aligned with multiple candidate code snippets, we are inspired by [22] to adopt the `MaxSim` operators during the training process. In addition, for fine-grained code, we conduct negative sample mining in the repository to enhance the contrastive learning process of fine-grained code snippets by adding additional in-function negative samples.

We validate our framework's effectiveness through extensive experiments across various datasets featuring different granularities of code search, and further experimental results demonstrate that MGS^3 possesses good interpretability. It is worth mentioning that our proposed approach is model-agnostic and can be initialized using existing pre-trained code representation models. In summary, our main contributions are summarized as follows:

- We explore a novel perspective on self-supervised code search, attempting to extract supervision signals between code snippets of various granularities and natural language comments from widely available repositories.
- We introduce a multi-granularity code search dataset, **MGCodeSearchNet**, which pairs various types of natural language comments with code snippets of differing granularities within repositories using heuristic methods.
- We propose MGS^3, a multi-granularity self-supervised code search framework. MGS^3 enhances structural information through HMGR, improves the representation of code snippets, and enhances contrastive learning by mining negative samples.
- We apply our framework to various pre-trained models and conduct experiments on a wide range of functional-level and snippet-level code search datasets. The results confirm that our method can improve the performance of existing pre-trained models and has good interpretability.

2 Related Work

2.1 Code Search

Due to the significant enhancement in model comprehension ability brought about by token-level pre-training tasks like Masked Language Modeling (MLM), CodeBERT [7] and CuBERT [21] have attempted to utilize the extensive programming language and natural language bimodal data in repositories for pre-training. GraphCodeBERT [9] attempts to introduce data flow graph signals on this basis to guide MLM pre-training, proving the importance of modeling data flows for code understanding. SynCoBERT [40] proposes a syntax-guided multimodal contrastive pre-training method that enhances the model's code representation capability by designing two new pre-training objectives: identifier prediction and AST edge prediction. UniXcoder [8] introduces a unified cross-modal pre-training model designed for programming languages. This model leverages cross-modal content such as Abstract Syntax Trees (AST) and code comments to enhance code representation, thereby improving performance in code search tasks.

Recently, some works have attempted to enhance code search tasks using contrastive learning methods [11, 12, 49]. ContraCode [20] categorizes existing code transformation techniques into three types: code compression, identifier renaming, and canonicalization transformations. It employs these techniques for data augmentation and utilizes the objectives of contrastive learning to distinguish between similar and dissimilar code snippets. CodeRetriever [29] attempts to combine unimodal and bimodal contrastive learning to train function-level code search models.

However, these methods overlook supervisory signals other than the function-level. We use a heuristic approach to align code snippets at multiple granularities with natural language comments from the repository, thereby enhancing the performance of the model on different granularity code search tasks.

2.2 Structured Code Representation

Source code is distinct from natural language as it encompasses abundant structural and semantic information. Consequently, to obtain structured code representation, some early works [1, 19, 42] treated source code as sequences or employed a Structure-Based Traversal (SBT) method [14] to flatten the Abstract Syntax Tree (AST), which allows the structural information within the program to enhance the representation of code. To better present the structure of the AST while maintaining the clarity of sequences, Hybrid-DeepCom [15] designed a novel structure-based traversal method to traverse the AST, which can explicitly recover the tree from the sequence generated by SBT and uses a hybrid attention component to merge lexical and syntactic information. CAST [37] proposed a method for hierarchical splitting and reconstruction of ASTs, aggregating the embeddings of the split subtrees to obtain the representation of the complete AST.

Subsequently, many researchers [2, 3, 23, 37] attempted to use structural information to guide the attention calculation process in Transformer networks. TPTrans [1] explored two representative path encoding methods and integrated them into a unified Transformer attention module by modifying the position encoding to include path encoding and explored their interactions. HiT [47] enhances the sequential representation of code by integrating a global, statement-level hierarchy with a local, token-level hierarchy. PA-former [5] attempted to model the relationships between phrases, tokens, sub-tokens, and their mappings, converting the source code into a pyramid representation, and applying a pyramid attention mechanism to effectively aggregate features across different levels.

2.3 Contrastive Learning Sample Construction

With the development of deep learning [30, 39], contrastive learning plays a key role in both unsupervised and self-supervised learning. Previous research [26, 34, 43] has highlighted the critical importance of enhancing the quality of positive and negative samples within the contrastive learning framework. Consequently, many researchers have sought better sampling strategies to improve search performance [43, 46]. In the realm of code search tasks, numerous studies have attempted to construct positive and negative samples for contrastive learning. For instance, Corder [4] uses various semantic-preserving code editing techniques for code refactoring to create positive samples. BOOST [6] introduces a structure-guided code transformation algorithm to generate positive samples that are functionally identical but structurally different. Li et al. [24] endeavors to construct representational-level positive samples that maintain semantic consistency without the need for additional data processing and training. Li et al. [25] introduces the Soft-InfoNCE technique to ameliorate the issue of false negatives in code search tasks and to explicitly differentiate the potential relevance of negative samples.

The existing research has primarily focused on constructing positive and negative samples at the function level, without exploring the issue of constructing negative samples within fine-grained code snippets. We propose a multi-granularity code search framework that identifies suitable negative samples for different granularities of code snippets to enhance the contrastive learning process.

3 The Dataset: MGCodeSearchNet

3.1 Data Collection

To gather and align code snippets of varying granularity from a wide range of open-source repositories, we leverage the existing

Table 1: The statistical results of our multi-granularity code search dataset.

Granularity	Ruby	JS	Go	Python	Java	PHP
Function-level	2.5K	5.8K	16.7K	25.2K	16.4K	24.1K
Block-level	1.7K	18.6K	20.7K	55.9K	47.6K	52.9K
Statement-level	8.5K	20.9K	45.8K	84.5K	24.8K	73.6K

CodeSearchNet corpus. The CodeSearchNet corpus [18] comprises an extensive collection of publicly available code data harvested from non-forked open-source GitHub repositories, all of which have licenses that are publicly accessible. It consists of 2.1 million pairs of data (functions paired with docstring) in six popular programming languages: Go, Java, JavaScript, PHP, Python, and Ruby. As described in [18], the docstring for the code is extracted from the function header comments. We adhere to the approach of [9] to filter out examples of low quality, resulting in 90.7K function-level code snippets.

To extract supervisory signals of varying granularity, we initially parse the functions within the codebase using the tree-sitter[2] library. Subsequently, we employ a heuristic approach based on programming conventions and code structure to align natural language comments with code snippets. Specifically, we first obtain natural language comments through parsing and then classify them according to Figure 1. Depending on the type, we employ different methods for alignment:

- **Docstrings**: we follow the practices established in prior research, aligning them with the entire function body.
- **Inline comments**: our focus is primarily on extracting snippets of code that contain control flows. We attempt to collect the subsequent code snippet based on the comment information; however, this heuristic method has a flaw that can lead to ambiguous alignment information. For instance, in Figure 1, it is unclear whether the natural language comment *"check if the file path exists"* should align with the *"if..."* block or the *"if...else..."* block. Our approach to handling this involves establishing a **one-to-many** alignment relationship between natural language comments and different code snippets. We then utilize semantic knowledge during the modeling process to make determinations.
- **Trailing comments**: we directly align these natural language comments with the code snippets of the current line.

Using the aforementioned heuristic alignment method, we can establish alignment relationships between various types of natural language comments and code snippets of multiple granularities.

3.2 Data Filtering

Due to the presence of substantial noise generate by programmers during the coding process within function comments, it may seriously damage the quality of the dataset. We attempt to utilize regular expressions to filter the extracted natural language comments. Specifically, we apply filtering in the following scenarios:

- Comments that include URL information (e.g., "https://...");
- Comments starting with special terms (e.g., "TODO");
- Comments used for automated code reviews (e.g., "Linter...");
- Comments that are shorter than four tokens.

In addition, by calculating the TF-IDF [35] scores of the current code snippet and the global code snippet, we will also filter out pairs that overly rely on the global code instead of the current fine-grained code snippet. The resulting collect MGCodeSearchNet includes 536K+ (query, code) pairs from 35K+ projects. The statistical information of the dataset is shown in Table 1. We present more statistical results of MGCodeSearchNet in Appendix A.

4 Preliminaries

4.1 Code Search

Code search is a technique for retrieving code snippets that are most relevant to a given query. During training, the model is trained by maximizing the similarity between the queries and the corresponding code. In the given space of (query, code snippet) pairs (Q, C), we denote $(q, c) \in (Q, C)$ as a pair. Here $q = \{q_1, q_2, ..., q_n\}$ represents a query composed of n tokens, and $c = \{c_1, c_2, \dots c_m\}$ represents a code snippet sequence composed of m tokens. We employ a query encoder E_Q and a code encoder E_C to encode the query q and the code c respectively. Subsequently, we train E_Q and E_C (in practice, parameters are typically shared between E_Q and E_C) to satisfy the following conditions for each query code pair (q, c):

$$\forall q \in Q, \max_{c \in C} f\left(q, c\right), \tag{1}$$

where $f(\cdot, \cdot)$ is used to calculate the matching score between a query and a code snippet. We use dot product to compute the similarity between query-code pairs. we employ the InfoNCE loss [33] for contrastive learning. Additionally, for efficient training, we adopt the in-batch negative strategy to construct negative sample pairs, aiming to minimize the similarity between them. Specifically, For a given batch of data, each query q can be paired with a positive sample code snippet c^+, as well as with $N-1$ other code snippets $\{c_1^-, c_2^-, \dots, c_{N-1}^-\}$ to create $N-1$ negative sample pairs, where N is the batch size. The InfoNCE loss can be described as follows:

$$\mathcal{L} = -\mathbb{E}\left[\log \frac{e^{f(q,c^+)/\tau}}{e^{f(q,c^+)/\tau} + \sum_{j=1}^{N-1} e^{f(q,c_j^-)/\tau}}\right], \tag{2}$$

where τ represents the temperature parameter in the contrastive learning loss function, which can generally be used to adjust the level of attention that the contrastive learning loss function pays to negative samples [10, 48, 50].

5 THE MGS^3 FRAMEWORK

5.1 Framework Overview

As depicted in Figure 2, to address the challenges mentioned earlier, we propose a **M**ulti-**G**rained **S**elf-**S**upervised Code **S**earch Framework (MGS^3) as a solution. Specifically, we use the existing code representation model as the backbone and MGCodeSearchNet as the training dataset. To leverage the structural information across different granularities for representing code snippets, we introduce a Hierarchical Multi-Grained Representation (HMGR) module in subsection 5.2. After aligning natural language comments with code snippets and obtaining representations of code at various granularities, we discuss the construction of positive and negative samples within the contrastive learning objective in subsection 5.3.

[2]https://tree-sitter.github.io

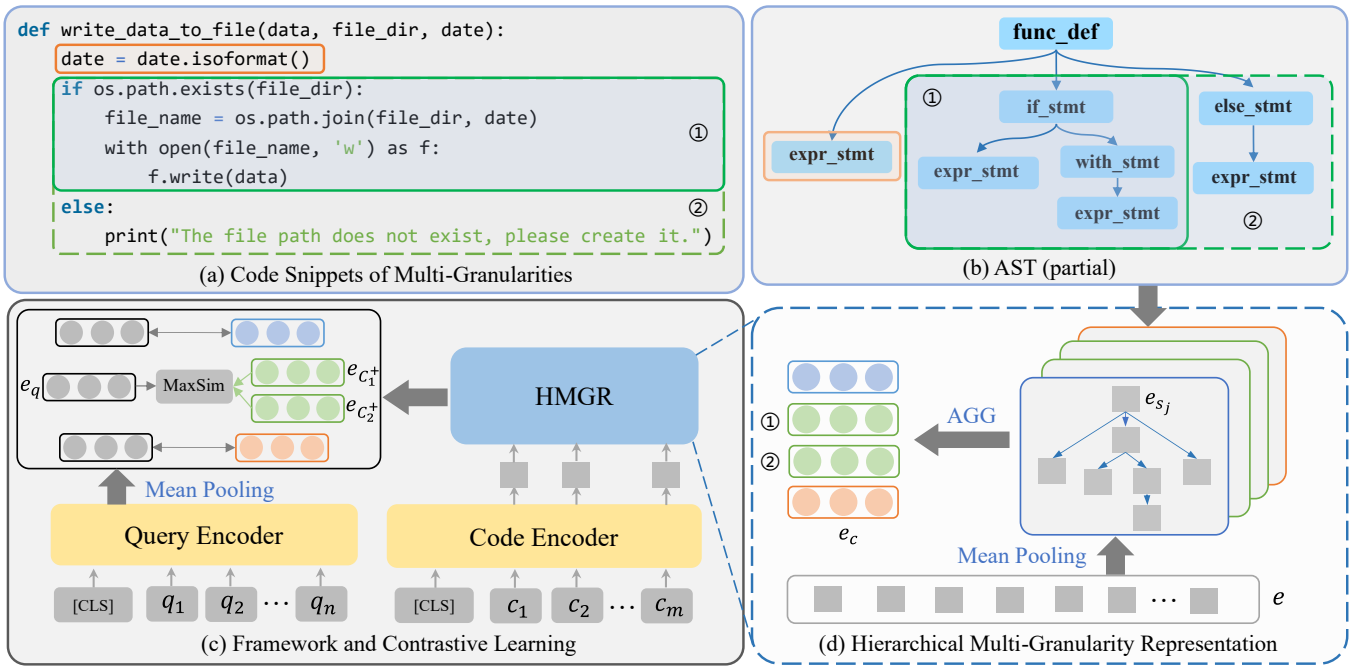


Figure 2: The overview of MGS^3. (a) Different granularity code snippets. (b) Depicts the abstract syntax tree (AST) corresponding to the code snippets, retaining only the statement and block nodes. (c) Illustrates the bi-encoder model using MaxSim for contrastive learning among code snippets. (d) The hierarchical aggregation in HMGR guided by AST.

5.2 Hierarchical Multi-Granularity Representation

Due to the need for representing code snippets at multiple granularities and the existence of structured relationships between different granularities within a function, we propose a Hierarchical Multi-Granularity Representation (HMGR) module. We consider code snippets at the statement-level granularity as the smallest unit of representation. For other coarser-grained code snippets, we attempt to aggregate their representations using hierarchical sub-node information within the code snippet. The difference from the previous method of using structured information from code is that we do not focus on more fine-grained token-level information, which reduces the complexity of the structure and provides a higher level of semantic guidance.

Since fine-grained code snippets contain contextual information within a function, extracting such code snippets directly from the function and aligning them with a query would result in the loss of contextual information. Therefore, regardless of the granularity of the code snippet being encoded, we treat the entire function as the input to the model and employ different extraction methods for different granularities of code snippets. Specifically, as shown in Figure 2 (c), we first encode the input code of the current code snippet using a code encoder model:

$$e = E_C([CLS], c_1, c_2, ..., c_n). \tag{3}$$

By encoding the function, we ultimately obtain the embeddings of all tokens in the function, i.e., $e = \{e_{cls}, e_1, e_2, ..., e_n\}$. After obtaining representations for all tokens in the function, we retrieve the representation of each token and use mean pooling to obtain the representation of all statement-level code snippets:

$$e_{S_j} = \frac{1}{|S_j|} \sum_{e_i \in S_j} e_i, \tag{4}$$

where S represents the collection of code snippets represented at the statement level, that is, $S = \{S_j\}_{j=1}^{|S|}$. For function-level and block-level code snippets, hierarchical relationships exist within these code snippets. Taking block-level code snippets as an example, typically, a block encompasses multiple statement-level code snippets and other block-level code snippets. Moreover, there are hierarchical relationships among these code snippets (e.g., in Figure 2 (a), the *"with..."* loop block includes the *"f.write(data)"* statement).

Therefore, during the modeling process of the current block-level code snippet, we consider aggregating the representations of its child nodes in a hierarchical manner. Specifically, as shown in Figure 2 (b), we first parse the function to obtain the abstract syntax tree (AST) and extract the subtree T corresponding to the current code snippet c. Then, as shown in Figure 2 (d), we utilize the AST to guide the aggregation of the statement-level representations:

$$\widehat{e}_c = AGG(e_c, \{e_v : v \in V(T) \text{ and } parent(v) = c\}), \tag{5}$$

where $V(T)$ signifies the set of nodes in the syntactic tree T. The function 'parent' is used to identify the parent node of a given node v in tree T. The AGG function represents an aggregation mechanism that combines the representation of the root node of the current

code snippet with the information from all its child nodes:

$$AGG(e_c, S) = \text{LayerNorm}(e_c + W * \frac{1}{|S|} \sum_{v \in S} e_v), \tag{6}$$

where W represents trainable parameters.

Due to code snippets in the codebase can be constructed during the offline phase, there is actually only a need to represent the query during the online stage. Consequently, a significant advantage of our proposed HMGR is that it does not increase the online latency.

5.3 Multi-Granularity Alignment

Our objective is to leverage the alignment information between comments and code snippets for contrastive learning. The essence of the contrastive learning lies in the construction of appropriate positive and negative samples. Therefore, we attempt to construct contrastive samples for code snippets of varying granularities.

5.3.1 ***Positive Sample Construction.*** In Section 3, we constructed positive sample pairs at various granularities through analytical construction. However, the use of heuristic algorithms for alignment can result in one natural language comment corresponding to multiple candidate block-level code snippets. Consequently, we aim to utilize the semantic information between natural language comments and candidate code snippets to select appropriate positive samples for contrastive learning.

Inspired by prior research [22], we employed a `MaxSim` operator for alignment. Specifically, as shown in the lower part of Figure 2 (c), for the natural language comment q and the potentially corresponding code snippet sets $C^+ = \{C_1^+, C_2^+, \cdots, C_n^+\}$, we calculate similarity scores using their embeddings. During contrastive learning, we select the code snippet with the highest score to align as the positive sample with the natural language comment:

$$f(q, C^+) = \max(e_q^\top e_{C_1^+}, \ldots, e_q^\top e_{C_n^+}), \tag{7}$$

where e_q is the encoding of the query. We obtain it by encoding the tokens of the query input into a model and then applying mean pooling. Additionally, we use HMGR from subsection 5.2 to obtain the code representation of $e_{C_i^+}$.

5.3.2 ***Negative Sample Construction.*** Since our constructed dataset contains multiple granularities, it is necessary to select appropriate negative samples for each granularity for contrastive learning. Previous research [25, 34] has already demonstrated that selecting as difficult negative samples as possible can facilitate the process of contrastive learning. Therefore, we choose negative samples of the same granularity for each code snippet during contrastive learning. For all granularities of code snippets, we can try to collect negative samples from other functions. Additionally, for block-level and statement-level code snippet samples, we have observed that these fine-grained code snippets allow for another source of negative samples — negative samples of corresponding granularity within the same function.

Specifically, MGS^3 extends the negative sample set c^- to include other code snippets of the same granularity within the same function. We consider that these internal negative samples from the same function can be considered more challenging than random samples from blocks and statements of other functions because they share similar function context information. This requires the model to have the ability to distinguish specific functionalities within the same function context. We summarize the full negative sample construction process in Appendix C.

5.4 Training Objective

MGS^3 uses three levels of supervision signals for contrastive learning training from natural language comments to code snippets. As mentioned in subsection 4.1, we utilize the InfoNCE loss function for pre-training contrastive learning between natural language comments and code snippets at different granularities. We denote the loss function at the function level as $\mathcal{L}_f$, at the block level as $\mathcal{L}_b$, and at the statement level as $\mathcal{L}_s$. Therefore, our final training objective $\mathcal{L}_{MG}$ is:

$$\mathcal{L}_{MG} = \mathcal{L}_f + \alpha \mathcal{L}_b + \beta \mathcal{L}_s, \tag{8}$$

where α and β are hyper-parameters to balance different granularity objective losses. The training goal is to minimize the integrated loss with respect to the model parameters.

6 Experiment

We conducted comprehensive experiments to answer the following research questions:

- **RQ1**: How does the MGS^3 perform across various granularities of code search tasks?
- **RQ2**: What is the adaptability of MGS^3 when fine-tuned for tasks involving multiple granularities of code search?
- **RQ3**: What are the roles of various modules in the MGS^3?
- **RQ4**: How interpretable is our proposed MGS^3?

6.1 Benchmark Datasets

To validate the performance of MGS^3 on code search tasks of different granularities, we evaluated it on several existing code search benchmarks of varying granularities:

- **Function level**: We select function-level evaluation benchmarks that are widely used in related work, including CodeSearchNet (CSN) [9, 18], Adv [32], CoSQA [16], and XLCoST-FL [51]. These datasets come from GitHub repositories, the StackOverflow[3] community, and the GeeksForGeeks[4].
- **Block level**: We choose SO-DS [13] and StaQC [44] and XLCoST-BL as block-level evaluation benchmarks, where SO-DS, StaQC are from Stackoverflow and XLCoST-BL is from GeeksForGeeks.
- **Statement level**: CoNaLa [45] collects question titles and replies from StackOverflow to construct a code search dataset.

We have summarized the detailed statistics of these benchmarks and dataset information in Appendix B.

6.2 Comparison Methods and Metrics

To validate that our propose MGS^3 framework can enhance the multi-granularity code search capabilities of existing pre-trained code representation models, we apply it to several models:

- **CodeBERT** [7] is a bimodal pre-trained model that is pre-trained through two tasks: Masked Language Modeling (MLM) and Replaced Token Detection (RTD).

[3]https://www.stackoverflow.com
[4]https://www.geeksforgeeks.org

Table 2: Under the zero-shot setting, we compare the original results of the pre-trained model with the performance using MGS^3 across various granularities benchmarks. We get the CodeT5+'s result by using the released checkpoint. Other results of compared models are reported by previous papers. All experiments meet the p<0.01 significance threshold.

Granularity	Dataset	CodeBERT		GraphCodeBERT		UniXCoder		CodeT5+	
		Original	w/ MGS^3	Original	w/ MGS^3	Original	w/ MGS^3	Original	w/ MGS^3
Function-level	Adv	0.5	**25.6**	0.5	**26.3**	2.4	**27.8**	28.5	**29.7**
	CoSQA	0.9	**36.6**	0.8	**37.7**	9.5	**39.9**	39.4	**41.5**
	XLCoST-FL	0.7	**26.0**	0.8	**26.4**	2.2	**28.3**	30.2	**33.7**
Block-level	SO-DS	0.4	**16.4**	0.6	**16.7**	0.5	**16.3**	9.2	**18.9**
	StaQC	0.6	**14.7**	0.5	**14.8**	0.7	**14.5**	8.3	**16.5**
	XLCoST-BL	0.9	**20.7**	1.2	**21.0**	0.8	**21.4**	13.4	**25.8**
Statement-level	CoNaLa	2.0	**12.8**	2.5	**13.1**	2.7	**12.6**	10.2	**15.0**

- **GraphCodeBERT** [9] proposes two structure-based pre-training tasks (data flow edge prediction and node alignment) to enhance code representation.
- **UniXcoder** [8] proposes to enhance code representation using cross-modal content such as AST and code comments.
- **CodeT5+** [41] uses a mix of pre-training objectives (span denoising, contrastive learning, etc.) for pre-training on monolingual and bilingual multi-language code corpora.

Following the previous research work, we use Mean Reciprocal Rank (MRR) [17] as the evaluation metric on all benchmarks.

$$MRR = \frac{1}{N}\sum_{i=1}^{N}\frac{1}{rank_i}, \tag{9}$$

where $rank_i$ is the rank of the correct code snippet related to the i-th query.

6.3 Implementation Details

Our MGS^3 framework is implemented in PyTorch[5]. We initialize the model using pre-trained code representation models, and for all models, we map the final output dimensions to 768. We employ the AdamW optimizer [31], experimenting with learning rates set to {1e-5, 2e-5, 5e-5}. The batch size is empirically set to 256. We select the number of training epochs from the set {20, 40, 60}, attempting to use an early stopping strategy. The maximum sequence lengths for the text and code are set to 128 and 320, respectively. We select the temperature τ in contrastive learning from the set {0.05, 0.1, 0.2}. We set the hyperparameters in Equation 8 as $\alpha = 1$ and $\beta = 0.6$. Detailed parameter sensitivity experiments are in Appendix D.3. The experiments described in this paper are conducted with three random seeds: 0, 1, and 2, and we will report the average results in the paper. All experiments meet the p<0.01 significance threshold. All experiments are performed on a Linux server equipped with four 2.30GHz Intel Xeon Gold 5218 CPUs and two Tesla A100 GPUs.

6.4 Zero-Shot Performance (RQ1)

First, to validate the performance of MGS^3 on various granularity code search tasks, we apply MGS^3 to multiple pre-trained code representation models and conduct contrastive learning pre-training on the MGCodeSearchNet dataset. Subsequently, we evaluate the models using various granularity code search benchmarks. Table 2 presents the zero-shot performance of different methods. It is worth noting that since the MGCodeSearchNet data is constructed based on CodeSearchNet, we do not show the results on CodeSearchNet under zero-shot conditions. By observation, we find that applying MGS^3 on pre-trained code representation models leads to performance improvements on all benchmarks. Furthermore, we observe that code representation models trained with token-level pretraining tasks perform poorly on all benchmarks, but achieve significant performance improvements after applying MGS^3. This demonstrates the effectiveness of our model in enhancing the performance of pretrained code representation models on code search tasks. Due to the inclusion of function-level code snippet contrastive learning during the pre-training phase, CodeT5+ exhibits better performance in function-level granularity benchmarks. However, it performs poorly in other granularity search benchmarks. After applying the MGS^3 framework, performance improvements are observed across all benchmarks, particularly in block-level and statement-level search benchmarks. This further validates the effectiveness of the MGS^3 framework.

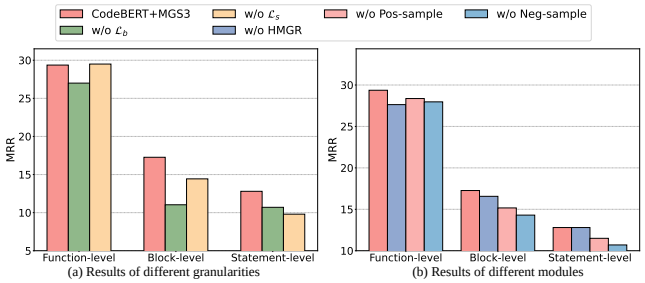


Figure 3: Results of ablation study on different granularities and modules. Full results can be found in Appendix D.2.

6.5 Fine-Tuning Performance (RQ2)

To validate the fine-tuning performance of the MGS^3 framework, we conduct fine-tuning on the training sets of various downstream benchmarks subsequent to the pre-training phase. We then evaluate the model's performance on the corresponding test sets. The

[5]Our code and dataset are available at https://github.com/smsquirrel/MGS3.

Table 3: In benchmarks of code search at different granularities, we compare the fine-tuning performance of pre-trained code representation models with the performance under a fine-tuning setting using MGS^3.

Granularity	Dataset	CodeBERT		GraphCodeBERT		UniXCoder		CodeT5+	
		Fine-tuning	w/ MGS^3	Fine-tuning	w/ MGS^3	Fine-tuning	w/ MGS^3	Fine-tuning	w/ MGS^3
Function-level	CSN	69.3	**71.4**	71.3	**73.2**	74.4	**75.7**	74.6	**75.6**
	Adv	27.2	**29.4**	35.2	**37.2**	41.3	**42.5**	43.3	**44.7**
	CoSQA	64.7	**66.0**	67.5	**68.8**	70.1	**71.3**	72.7	**73.9**
	XLCoST-FL	58.0	**60.7**	59.4	**62.2**	59.1	**61.9**	63.3	**65.1**
Block-level	SO-DS	23.1	**27.3**	25.3	**27.9**	23.6	**27.3**	26.1	**29.5**
	StaQC	23.4	**27.6**	23.8	**28.7**	23.1	**28.2**	25.7	**29.4**
	XLCoST-BL	30.9	**38.9**	31.6	**39.5**	30.2	**38.0**	35.8	**40.0**
Statement-level	CoNaLa	20.9	**25.0**	23.5	**26.9**	20.4	**25.1**	22.8	**27.1**

experimental results are presented in Table 3. Observation of these results reveals that, compared to other baselines, the application of the MGS^3 framework significantly enhances performance on the CodeSearchNet dataset. This suggests that our method effectively leverages a broader range of supervisory signals within repositories to improve model performance. Furthermore, our approach achieves the best performance across various granularity search benchmarks, demonstrating that models pre-trained with the MGS^3 framework can be rapidly adapted to downstream code search tasks of different granularities. Further observation shows that our model exhibits significantly better performance on the XLCoST-BL dataset, which may be attributed to the close similarity in data distribution between XLCoST-BL and our training dataset. It is also observed that our method demonstrates a more pronounced improvement on smaller training sets (such as CoNaLa), suggesting that our approach holds substantial potential for application in scenarios characterized by data sparsity.

6.6 Ablation Study (RQ3)

To examine the effectiveness of incorporating alignment signals of varying granularities and the role of each module within MGS^3, we conduct a series of ablation studies. We initialize our framework using the CodeBERT model and evaluate its performance across code search tasks of multiple granularities.

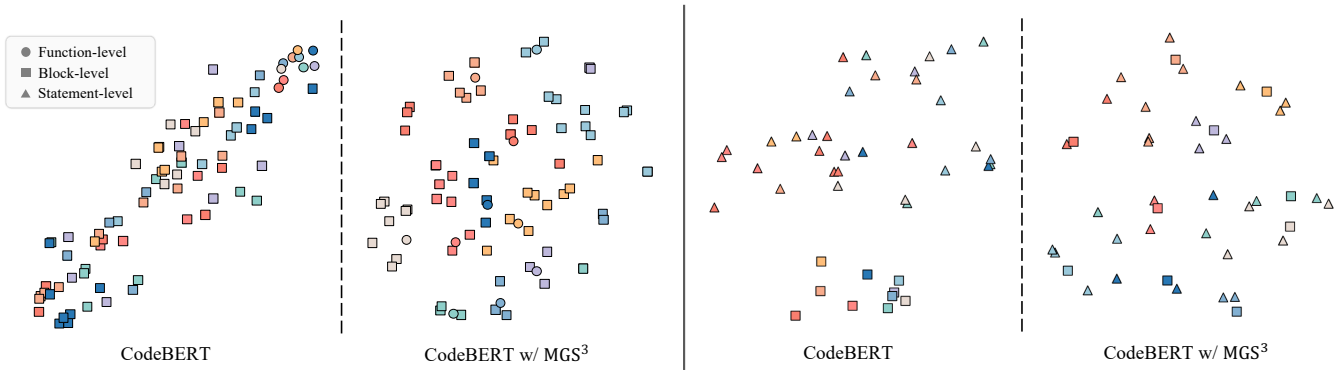
6.6.1 ***Effectiveness of Different Granularities.*** To investigate the impact of alignment signals at different granularities on the model, we conduct an ablation study. Specifically, we introduce the conditions "w/o $\mathcal{L}_b$" and "w/o $\mathcal{L}_s$," which exclude the loss terms for block-level and statement-level alignment signals, respectively, from Equation 8. The results, as shown in Figure 3 (a), indicate a noticeable decline in model performance on code search tasks corresponding to these granularities. This suggests that adding supervisory signals at specific granularities can effectively aid the code search model in adapting to code search tasks at those levels. Furthermore, the removal of $\mathcal{L}_b$ results in reduced performance on code search tasks at other granularities, implying that block-level alignment signals can enhance the model's understanding of code snippets at various granularities. Conversely, the elimination of $\mathcal{L}_s$ appears to improve performance on function-level code search tasks. We consider that this is due to significant differences between statement-level and function-level code snippets.

6.6.2 ***Effectiveness of Different Modules.*** To validate the contribution of each module within the MGS^3, We denote "w/o HMGR" to indicate the removal of the HMGR module and instead use mean pooling to obtain representations of code snippets at different granularities. "w/o Pos-sample" indicates the removal of alignment and instead aligning with the nearest largest block in natural language comments. "w/o Neg-sample" indicates the removal of in-function negative samples and only using in-batch negative samples for contrastive learning.

The final experimental results are shown in Figure 3 (b), removing any module will lead to a decrease in performance on downstream tasks, confirming the effectiveness of our proposed modules. It is worth noting that we observe a decrease in performance on code search tasks at all granularities when removing the HMGR module, especially at the block-level and function-level. This suggests that the HMGR module helps models utilize structured information between different granularity code snippets. Additionally, we notice that "w/o Pos-sample" significantly reduces the performance of block-level code search tasks. This implies that this module contributes to more accurate and effective block-level alignment and proves that aligning natural language comments with code snippets solely through heuristic methods may result in suboptimal performance. Finally, we observe that "w/o Neg-sample" leads to a decrease in performance for both block-level and statement-level code search tasks. This indicates that in-function negative samples can increase the difficulty for fine-grained code snippet negatives and provide more diverse sources of negative samples, resulting in better contrastive learning outcomes.

6.7 Visualization (RQ4)

6.7.1 ***Representation Distribution of Code Snippets with Different Granularities.*** In this section, we aim to visually demonstrate the capabilities of our framework. We first randomly selected 10 functions from the XLCoST dataset and parsed them to obtain the corresponding block-level code snippets, as well as collected 10 block-level code snippets and parsed them to obtain the corresponding statement-level code snippets. Then we will encode the selected code snippets using different models. Finally, we use t-SNE [38] to project the representations of code snippets of different granularities into a two-dimensional space. We marked snippets belonging to the same function with identical colors and used different shapes to denote different granularities: triangles for the function level,



(a) Visualization of function-level and block-level (b) Visualization of block-level and statement-level

Figure 4: Visualization of code snippets represented in multiple granularities. Code snippets with the same functionality are identified with the same color, while code snippets of different granularities are identified with different shapes.

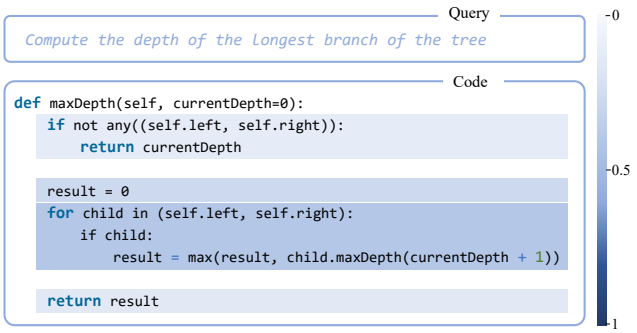


Figure 5: Visualize the importance of final search scores for different granularity code snippets.

squares for the block level, and circles for the statement level. As can be observed from Figure 4 (a), the visualization of the original CodeBERT representations shows function-level code snippets clustered together while block-level snippets are dispersed. After applying the MGS^3 framework, the representations of code snippets from the same function but of different granularities appear more concentrated, and a similar phenomenon is evident in Figure 4 (b). This indicates that our framework is capable of achieving superior representations of code snippets across different granularities.

6.7.2 ***Interpretability Verification.*** To understand whether MGS^3 can leverage fine-grained representations to enhance coarse-grained search tasks, we attempted to conduct an interpretability analysis on MGS^3. Specifically, we obtain queries and corresponding function-level code snippets from the COSQA dataset, and determine their importance by calculating the contribution of fine-grained code snippets within functions to the final search score. To achieve this goal, we utilized the Grad-CAM [36] method to visualize the gradients of fine-grained code representations relative to the final search score. Figure 5 displays the visualization results. In the analyzed function, a for loop used to traverse tree nodes and calculate the maximum depth was identified as an important part of understanding the function's semantics and matching it with the query. Observation reveals that this *"for..."* loop made the most significant contribution to the final search score. The visualization results confirm that MGS^3 can utilize information from code snippets of varying granularities within a function to assist in function-level code search, and demonstrate that MGS^3 has good interpretability.

7 Conclusion

In this paper, we aimed to efficiently leverage self-supervised signals from repositories to enhance model performance across various code search scenarios. We created a dataset, MGCodeSearchNet, which pairs natural language comments with code snippets at different granularities. We then introduced MGS^3, a novel multi-granularity code search framework that improves code representation by modeling the hierarchical relationships between code snippets. MGS^3 also enhances contrastive learning by constructing positive and negative samples from code snippets of varying granularities. Our experiments on multi-granularity code search benchmarks demonstrated MGS^3's superior representation capabilities. Additionally, analytical experiments showed that our method improves the interpretability of pre-trained models in code search tasks. In future work, we plan to expand the MGCodeSearchNet dataset using a wider range of online repositories and to attempt to combine MGS^3 with retrieval-augmented generation techniques to achieve better code generation performance.

Acknowledgments

This research was partially supported by grants from the National Natural Science Foundation of China (Grants No. 62337001, 62477044), the Key Technologies R & D Program of Anhui Province (No. 202423k09020039), the Fundamental Research Funds for the Central Universities and the Iflytek joint research program.

References

[1] Wasi Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. 2020. A Transformer-based Approach for Source Code Summarization. In *Proceedings of*

the 58th Annual Meeting of the Association for Computational Linguistics. 4998–5007.

[2] Miltiadis Allamanis, Marc Brockschmidt, and Mahmoud Khademi. 2018. Learning to Represent Programs with Graphs. In *International Conference on Learning Representations*. https://openreview.net/forum?id=BJOFETxR-

[3] Uri Alon, Shaked Brody, Omer Levy, and Eran Yahav. 2018. code2seq: Generating Sequences from Structured Representations of Code. In *International Conference on Learning Representations*.

[4] Nghi DQ Bui, Yijun Yu, and Lingxiao Jiang. 2021. Self-supervised contrastive learning for code retrieval and summarization via semantic-preserving transformations. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 511–521.

[5] Lei Chai and Ming Li. 2022. Pyramid Attention For Source Code Summarization. *Advances in Neural Information Processing Systems* 35 (2022), 20421–20433.

[6] Yangruibo Ding, Luca Buratti, Saikat Chakraborty, Saurabh Pujar, Alessandro Morari, and Baishakhi Ray. 2021. Contrastive learning for source code with structural and functional properties. (2021).

[7] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020, Online Event, 16-20 November 2020 (Findings of ACL, Vol. EMNLP 2020)*. 1536–1547. doi:10.18653/v1/2020.findings-emnlp.139

[8] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. 2022. UniXcoder: Unified Cross-Modal Pre-training for Code Representation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2022, Dublin, Ireland, May 22-27, 2022*. 7212–7225. doi:10.18653/v1/2022.acl-long.499

[9] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, Michele Tufano, Shao Kun Deng, Colin B. Clement, Dawn Drain, Neel Sundaresan, Jian Yin, Daxin Jiang, and Ming Zhou. 2021. GraphCodeBERT: Pre-training Code Representations with Data Flow. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. https://openreview.net/forum?id=jLoC4ez43PZ

[10] Kaiming He, Haoqi Fan, Yuxin Wu, Saining Xie, and Ross Girshick. 2020. Momentum contrast for unsupervised visual representation learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 9729–9738.

[11] Liyang He, Zhenya Huang, Enhong Chen, Qi Liu, Shiwei Tong, Hao Wang, Defu Lian, and Shijin Wang. 2023. An efficient and robust semantic hashing framework for similar text search. *ACM Transactions on Information Systems* 41, 4 (2023), 1–31.

[12] Liyang He, Zhenya Huang, Chenglong Liu, Rui Li, Runze Wu, Qi Liu, and Enhong Chen. 2024. One-bit deep hashing: Towards resource-efficient hashing model with binary neural network. In *Proceedings of the 32nd ACM International Conference on Multimedia*. 7162–7171.

[13] Geert Heyman and Tom Van Cutsem. 2020. Neural code search revisited: Enhancing code snippet retrieval through natural language intent. *arXiv preprint arXiv:2008.12193* (2020).

[14] Xing Hu, Ge Li, Xin Xia, David Lo, and Zhi Jin. 2018. Deep code comment generation. In *Proceedings of the 26th conference on program comprehension*. 200–210.

[15] Xing Hu, Ge Li, Xin Xia, David Lo, and Zhi Jin. 2020. Deep code comment generation with hybrid lexical and syntactical information. *Empirical Software Engineering* 25 (2020), 2179–2217.

[16] Junjie Huang, Duyu Tang, Linjun Shou, Ming Gong, Ke Xu, Daxin Jiang, Ming Zhou, and Nan Duan. 2021. CoSQA: 20,000+ Web Queries for Code Search and Question Answering. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. 5690–5700.

[17] David A Hull. 1999. Xerox TREC-8 Question Answering Track Report.. In *TREC*.

[18] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2019. Codesearchnet challenge: Evaluating the state of semantic code search. *arXiv preprint arXiv:1909.09436* (2019).

[19] Srinivasan Iyer, Ioannis Konstas, Alvin Cheung, and Luke Zettlemoyer. 2016. Summarizing source code using a neural attention model. In *54th Annual Meeting of the Association for Computational Linguistics 2016*. Association for Computational Linguistics, 2073–2083.

[20] Paras Jain, Ajay Jain, Tianjun Zhang, Pieter Abbeel, Joseph Gonzalez, and Ion Stoica. 2021. Contrastive Code Representation Learning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. 5954–5971.

[21] Aditya Kanade, Petros Maniatis, Gogul Balakrishnan, and Kensen Shi. 2020. Learning and Evaluating Contextual Embedding of Source Code. In *International Conference on Machine Learning*. 5110–5121.

[22] Omar Khattab and Matei Zaharia. 2020. Colbert: Efficient and effective passage search via contextualized late interaction over bert. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*. 39–48.

[23] Alexander LeClair, Sakib Haque, Lingfei Wu, and Collin McMillan. 2020. Improved code summarization via a graph neural network. In *Proceedings of the 28th international conference on program comprehension*. 184–195.

[24] Haochen Li, Chunyan Miao, Cyril Leung, Yanxian Huang, Yuan Huang, Hongyu Zhang, and Yanlin Wang. 2022. Exploring Representation-level Augmentation for Code Search. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. 4924–4936.

[25] Haochen Li, Xin Zhou, Anh Luu, and Chunyan Miao. 2023. Rethinking Negative Pairs in Code Search. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 12760–12774.

[26] Mingjia Li, Hong Qian, Jinglan Lv, Mengliang He, Wei Zhang, and Aimin Zhou. 2025. Foundation model enhanced derivative-free cognitive diagnosis. *Frontiers of Computer Science* 19, 1 (2025), 191318.

[27] Rui Li, Liyang He, Qi Liu, Yuze Zhao, Zheng Zhang, Zhenya Huang, Yu Su, and Shijin Wang. 2024. CONSIDER: Commonalities and Specialties Driven Multilingual Code Retrieval Framework. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 8679–8687.

[28] Rui Li, Qi Liu, Liyang He, Zheng Zhang, Hao Zhang, Shengyu Ye, Junyu Lu, and Zhenya Huang. 2024. Optimizing Code Retrieval: High-Quality and Scalable Dataset Annotation through Large Language Models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 2053–2065.

[29] Xiaonan Li, Yeyun Gong, Yelong Shen, Xipeng Qiu, Hang Zhang, Bolun Yao, Weizhen Qi, Daxin Jiang, Weizhu Chen, and Nan Duan. 2022. CodeRetriever: A Large Scale Contrastive Pre-Training Method for Code Search. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. 2898–2910.

[30] Qi Liu, Zhenya Huang, Yu Yin, Enhong Chen, Hui Xiong, Yu Su, and Guoping Hu. 2019. Ekt: Exercise-aware knowledge tracing for student performance prediction. *IEEE Transactions on Knowledge and Data Engineering* 33, 1 (2019), 100–115.

[31] Ilya Loshchilov and Frank Hutter. 2017. Fixing Weight Decay Regularization in Adam. *CoRR* abs/1711.05101 (2017). arXiv:1711.05101 http://arxiv.org/abs/1711.05101

[32] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin B. Clement, Dawn Drain, Daxin Jiang, Duyu Tang, Ge Li, Lidong Zhou, Linjun Shou, Long Zhou, Michele Tufano, Ming Gong, Ming Zhou, Nan Duan, Neel Sundaresan, Shao Kun Deng, Shengyu Fu, and Shujie Liu. 2021. CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation.

[33] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. 2018. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748* (2018).

[34] Yingqi Qu, Yuchen Ding, Jing Liu, Kai Liu, Ruiyang Ren, Wayne Xin Zhao, Daxiang Dong, Hua Wu, and Haifeng Wang. 2021. RocketQA: An Optimized Training Approach to Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 5835–5847.

[35] Juan Ramos et al. 2003. Using tf-idf to determine word relevance in document queries. In *Proceedings of the first instructional conference on machine learning*, Vol. 242. Citeseer, 29–48.

[36] Ramprasaath R Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. 2017. Grad-cam: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE international conference on computer vision*. 618–626.

[37] Ensheng Shi, Yanlin Wang, Lun Du, Hongyu Zhang, Shi Han, Dongmei Zhang, and Hongbin Sun. 2021. CAST: Enhancing Code Summarization with Hierarchical Splitting and Reconstruction of Abstract Syntax Trees. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. 4053–4062.

[38] Laurens van der Maaten and Geoffrey E. Hinton. 2008. Visualizing Data using t-SNE. *Journal of Machine Learning Research* 9 (2008), 2579–2605. https://api.semanticscholar.org/CorpusID:5855042

[39] Fei Wang, Qi Liu, Enhong Chen, Zhenya Huang, Yu Yin, Shijin Wang, and Yu Su. 2022. NeuralCD: a general framework for cognitive diagnosis. *IEEE Transactions on Knowledge and Data Engineering* 35, 8 (2022), 8312–8327.

[40] Xin Wang, Yasheng Wang, Fei Mi, Pingyi Zhou, Yao Wan, Xiao Liu, Li Li, Hao Wu, Jin Liu, and Xin Jiang. 2021. Syncobert: Syntax-guided multi-modal contrastive pre-training for code representation. *arXiv preprint arXiv:2108.04556* (2021).

[41] Yue Wang, Hung Le, Akhilesh Deepak Gotmare, Nghi DQ Bui, Junnan Li, and Steven CH Hoi. 2023. Codet5+: Open code large language models for code understanding and generation. *arXiv preprint arXiv:2305.07922* (2023).

[42] Bolin Wei, Ge Li, Xin Xia, Zhiyi Fu, and Zhi Jin. 2019. Code generation as a dual task of code summarization. *Advances in neural information processing systems* 32 (2019).

[43] Lee Xiong, Chenyan Xiong, Ye Li, Kwok-Fung Tang, Jialin Liu, Paul N Bennett, Junaid Ahmed, and Arnold Overwijk. 2020. Approximate Nearest Neighbor Negative Contrastive Learning for Dense Text Retrieval. In *International Conference on Learning Representations*.

[44] Ziyu Yao, Daniel S Weld, Wei-Peng Chen, and Huan Sun. 2018. Staqc: A systematically mined question-code dataset from stack overflow. In *Proceedings of the 2018 World Wide Web Conference*. 1693–1703.

[45] Pengcheng Yin, Bowen Deng, Edgar Chen, Bogdan Vasilescu, and Graham Neubig. 2018. Learning to mine aligned code and natural language pairs from stack overflow. In *Proceedings of the 15th international conference on mining software repositories*. 476–486.
[46] Hang Zhang, Yeyun Gong, Yelong Shen, Jiancheng Lv, Nan Duan, and Weizhu Chen. 2021. Adversarial Retriever-Ranker for Dense Text Retrieval. In *International Conference on Learning Representations*.
[47] Kechi Zhang, Zhuo Li, Zhi Jin, and Ge Li. 2023. Implant Global and Local Hierarchy Information to Sequence based Code Representation Models. *arXiv preprint arXiv:2303.07826* (2023).
[48] Zheng Zhang, Qi Liu, Zirui Hu, Yi Zhan, Zhenya Huang, Weibo Gao, and Qingyang Mao. 2024. Enhancing Fairness in Meta-learned User Modeling via Adaptive Sampling. In *Proceedings of the ACM on Web Conference 2024*. 3241–3252.
[49] Zheng Zhang, Qi Liu, Hao Jiang, Fei Wang, Yan Zhuang, Le Wu, Weibo Gao, and Enhong Chen. [n. d.]. Fairlisa: Fair user modeling with limited sensitive attributes information. In *Thirty-seventh Conference on Neural Information Processing Systems*.
[50] Hongke Zhao, Chuang Zhao, Xi Zhang, Nanlin Liu, Hengshu Zhu, Qi Liu, and Hui Xiong. 2023. An ensemble learning approach with gradient resampling for class-imbalance problems. *INFORMS Journal on Computing* 35, 4 (2023), 747–763.
[51] Ming Zhu, Aneesh Jain, Karthik Suresh, Roshan Ravindran, Sindhu Tipirneni, and Chandan K Reddy. 2022. Xlcost: A benchmark dataset for cross-lingual code intelligence. *arXiv preprint arXiv:2206.08474* (2022).

A MGCodeSearchNet details

We have compiled the statistics of the current dataset. The average word count of code comments is 14.9. The average number of lines and tokens (using CodeBERT's tokenizer) for different granularity of code segments is shown in Table 4.

	Function-level	Block-level	Statement-level
Lines	14.2	5.7	1
Tokens	122.2	51.3	15.7

Table 4: Comparison of lines and tokens at different levels

B Benchmark Statictics

To validate the performance of MGS^3 on code search tasks of different granularities, we evaluated it on several existing code search benchmarks of varying granularities:

- **Function level**: CodeSearchNet (CSN) [9, 18] collects datasets from six different programming languages from GitHub repositories, which can be used to assess model performance across different programming languages. Adv [32] is based on the CodeSearch dataset and has normalized method and variable names in the development/test sets to make it more challenging. CoSQA [16] queries are collected from web search engines, thus better verifying the model's performance in real code search scenarios. XLCoST-FL [51] has gathered programming questions along with corresponding programs in seven different programming languages from the GeeksForGeeks[6].
- **Block level**: SO-DS [13] and StaQC [44] are collected from StackOverflow[7] questions, which can validate the model's code search performance in programming communities like StackOverflow. XLCoST-BL is a block-level aligned code search dataset obtained through the division of function-level comments in XLCoST. This relies on the well-defined comment specifications on the GeeksForGeeks.
- **Statement level**: CoNaLa [45] collects question titles and replies from StackOverflow to construct a code search dataset.

The statistical information of the benchmark is shown in Table 5.

Table 5: The statistics of benchmark datasets.

Dataset	Granularity	Training	Validation	Test
CSN-Ruby	Function	2.5K	1.4K	1.2K
CSN-JS	Function	5.8K	3.9K	3.3K
CSN-Go	Function	16.7K	7.3K	8.1K
CSN-Python	Function	25.2K	13.9K	14.9K
CSN-Java	Function	16.4K	5.2K	10.9K
CSN-PHP	Function	24.1K	13.0K	14.0K
Adv	Function	28.0K	9.6K	19.2K
CoSQA	Function	19.0K	0.5K	0.5K
XLCoST-FL	Function	9.2K	0.5K	0.9K
SO-DS	Block	14.2K	0.9K	1.1K
StaQC	Block	20.4K	2.6K	2.7K
XLCoST-BL	Block	81.2K	3.9K	7.3K
CoNaLa	Statement	2.8K	-	0.8K

C Negative Sample Selection

MGS^3 not only uses in-batch negative samples but also adds other code snippets of the same granularity within the same function to expand the negative sample set c^-. We consider that these internal negative samples from the same function can be considered more challenging than random samples from blocks and statements of other functions because they share similar function context information. This requires the model to have the ability to distinguish specific functionalities within the same function context. It is worth noting that we do not consider code snippets with a nesting relationship within a function as in-function negatives, due to their often close semantic connections. Instead, we choose code snippets that are mutually independent as in-function negatives.

D Extra of Experiment

D.1 Results of the Fine-tuning Experiment.

We present the results of the fine-tuning in Table 6. By observing the experimental results on the CodeSearchNet dataset across various programming languages, it can be seen that by applying MGS^3 method, significant improvements can be achieved in all programming languages, which demonstrates the effectiveness of the MGS^3.

D.2 Ablation Study

We attempted to present the complete ablation study experiment in Table 7. We can observe that different granular data sets exhibit relatively similar changes for different ablation experiment settings. In the block-level experimental results, we observed a significant performance drop in the SO-DS benchmark after deleting $\mathcal{L}_s$. We speculate that this is because the benchmark comes from StackOverflow, and during the collection process, some statement-level code snippets were mixed in.

[6]https://www.geeksforgeeks.org
[7]https://www.stackoverflow.com

Table 6: In benchmarks of code search at different granularities, we compare the fine-tuning performance of pre-trained code representation models with the performance under a fine-tuning setting using MGS^3.

Granularity	Dataset	CodeBERT		GraphCodeBERT		UniXCoder		CodeT5+	
		Fine-tuning	w/ MGS^3	Fine-tuning	w/ MGS^3	Fine-tuning	w/ MGS^3	Fine-tuning	w/ MGS^3
Function-level	CSN-Ruby	67.9	**70.7**	70.3	**72.5**	74.0	**75.7**	74.9	**76.0**
	CSN-JS	62.0	**64.0**	64.4	**66.6**	68.4	**69.1**	69.2	**69.8**
	CSN-Go	88.2	**89.9**	89.7	**91.3**	91.5	**92.1**	91.0	**91.8**
	CSN-Python	67.2	**69.4**	69.2	**70.7**	72.0	**73.6**	71.9	**72.8**
	CSN-Java	67.6	**69.5**	69.1	**71.3**	72.6	**74.8**	72.4	**73.6**
	CSN-PHP	62.8	**64.7**	64.9	**66.5**	67.6	**69.0**	68.2	**69.3**
	Adv	27.2	**29.4**	35.2	**37.2**	41.3	**42.5**	43.3	**44.7**
	CoSQA	64.7	**66.0**	67.5	**68.8**	70.1	**71.3**	72.7	**73.9**
	XLCoST-FL	58.0	**60.7**	59.4	**62.2**	59.1	**61.9**	63.3	**65.1**
Block-level	SO-DS	23.1	**27.3**	25.3	**27.9**	23.6	**27.3**	26.1	**29.5**
	StaQC	23.4	**27.6**	23.8	**28.7**	23.1	**28.2**	25.7	**29.4**
	XLCoST-BL	30.9	**38.9**	31.6	**39.5**	30.2	**38.0**	35.8	**40.0**
Statement-level	CoNaLa	20.9	**25.0**	23.5	**26.9**	20.4	**25.1**	22.8	**27.1**

Table 7: The results of ablation experiments conducted on different granularity code search benchmarks.

		Function-level			Block-level			Statement-level
		Adv	CoSQA	XLCoST-FL	SO-DS	StaQC	XLCoST-BL	CoNaLa
CodeBERT+MGS3		25.6	36.6	25.9	16.4	14.7	20.7	12.8
Granularity	w/o $\mathcal{L}_b$	23.2	34.2	23.6	9.5	8.8	14.8	10.7
	w/o $\mathcal{L}_s$	25.5	36.7	26.3	13.4	13.0	16.9	9.8
Module	w/o HMGR	23.8	35.2	23.9	15.4	14.6	19.7	12.8
	w/o Pos-sample	24.6	35.7	24.8	14.2	13.2	18.1	11.5
	w/o Neg-sample	24.3	35.1	24.5	12.9	12.7	17.3	10.7

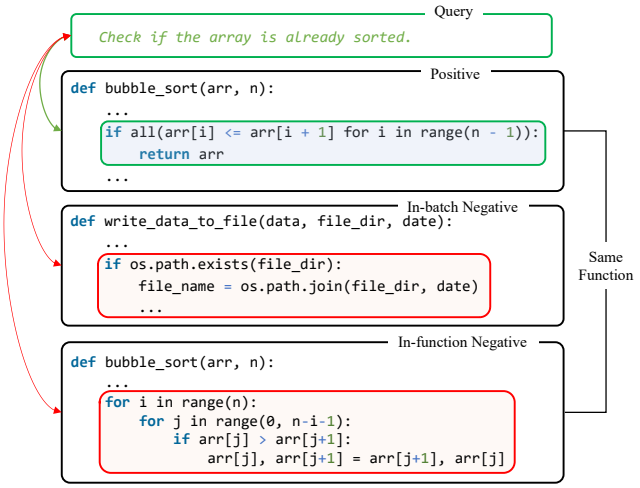


Figure 6: We employed methods for acquiring negative samples of fine-grained code snippets, which include in-batch negatives and in-function negatives, taking block-level code snippets as an example.

D.3 Parameter Sensitivity Experiments

For the weights α and β in the loss functions of different granularities, we conducted parameter sensitivity experiments using the CodeBERT model on Adv, StaQC, and CoNaLa. Table 6 shows the performance of α and β at 0.2, 0.4, 0.6, 0.8, and 1. When adjusting one of the parameters, we set the other to 0 to facilitate the observation of results.

Table 8: Results of sensitivity experiments on α and β.

α	Adv	StaQC	CoNaLa	β	Adv	StaQC	CoNaLa
0.2	24.2	4.0	3.5	0.2	**24.2**	3.2	5.7
0.4	24.5	7.8	5.9	0.4	24.0	7.1	8.9
0.6	25.1	10.2	7.4	0.6	23.7	**9.0**	10.0
0.8	**25.7**	11.6	8.5	0.8	23.0	8.7	10.2
1.0	25.5	**13.0**	**9.8**	1.0	23.2	8.8	**10.6**